

2026 오픈소스 개발자대회 결과보고서

작품명: **CAPNET**

접수번호: 915

팀명: 지엔

팀장: 서우석

제출일: 2026. 08. 25

구분	작성 항목	제출 구분	작성 분량
결과보고서	프로젝트 개요 및 세부 내용	필수	5페이지 이내
붙임1	SBOM(소프트웨어 자재명세서)	필수	제한 없음
붙임2	AI 모델 활용 정보	해당 시	제한 없음

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	지엔	팀 인원 (팀장 포함)	3명
참가부문	일반	과제유형	자유과제

□ 결과보고서

프로젝트 개요	
프로젝트명	CapNet
프로젝트 등록 URL	https://github.com/gncorpseo-commits/capnet
시연영상	https://youtu.be/RjFiGpmLTbk
프로젝트 소개	CapNet - AI 실행 자원을 능력단위로 요청하는 오픈소스 실행 계층
프로젝트 세부 내용	
개발배경 및 목적	【개발배경 및 목적】 AI 서비스를 구축할 때 일반적으로는 특정 GPU·추론 서버를 지정하고 그 API를 호출한다. 서버가 소수일 때는 단순하지만, 수요가 늘면 증설·관리와 자원 활용이 문제가 된다. 한편 조직과 연구환경에는 업무·상황에 따라 유휴로 남는 GPU PC, 워크스테이션, 서버 등 다양한 연산 자원이 있다. 문제는 자원을 단순히 연결하는 것이 아니라, 어떤 작업을 어떤 자원에서 실행해도 되는지를 판단하고 그 실행 경로를 관리하는 것이다. CapNet은 AI 실행 자원을 서버 주소가 아니라 하나의 실행 계층으로 추상화하는 것에서 출발했다. 사용자는 “어느 GPU에서 실행할 것인가” 대신 <code>image.classify</code> 같은 능력을 요청하고, Core가 승인된 자원 중 신뢰 도메인·연산 등급·Agent 자격·가용성을 만족하는 대상을 고른다. 서비스와 실행 자원의 결합을 분리해, 자원이 추가·교체돼도 호출자는 서버 주소와 구현을 직접 관리하지 않는다. 배정뿐 아니라 어떤 Agent가 어떤 Node에서 어떤 조건으로 실행됐는지 증적을 남기며, 신뢰 경계를 벗어나는 배정은 애플리케이션이 아니라 PostgreSQL 제약이 거절한다. 이번 MVP는 이 실행 계층의 핵심을 조직 내부의 승인된 Node 플릿에서 구현·검증했다. 향후에는 초청 유휴 자원, 신뢰 모델이 확보된 외부 자원으로 확장할 수 있도록 설계한다.

개발환경	【개발환경】 - HW: 팀 노트북 CPU · Docker Desktop / WSL2 - SW: Python 3.11 · PostgreSQL 16 · FastAPI · PyTorch 2.13.0+cpu · torchvision 0.28.0+cpu · safetensors - 실행: Docker Compose (postgres 1 + Core 1 + Node 3) - 데이터: EuroSAT RGB (Zenodo 7711810, MIT) — 원본 미동봉, 다운로드 스크립트 제공 - 모델: 사전학습 가중치 미사용. EuroSAT scratch 학습만 - 외부 상용 AI API 호출 없음 — 추론 전량 로컬 컨테이너
시스템 구성 및 아키텍처	【시스템 구성 및 아키텍처】 - Node: 실행 가능 컴퓨터. Core가 승인한 작업만 수행 (자기 등급 주장 불가) - Agent: 특정 AI 능력의 구현 - Capability: 사용자가 요구하는 작업의 계약 - Core(:8000): Capability × Agent × Node 조합으로 배정·증적. 사용자 접점 - postgres: 라우팅·게이트 불변식(복합 FK·호환 행렬·증서) - 연결: Node → Core outbound(HTTP polling). WebSocket은 향후 확장 - 흐름: 사용자→Core(능력만 요청) → Core 배정 → Node가 배정 가져와 실행 → 결과·증적 조회 - 무단 호출 거부(HTTP 403). 사용자는 기기 주소를 모른다
프로젝트 주요기능	【프로젝트 주요기능】 실행 자원 네트워크. 기존에는 서비스가 특정 실행 서버를 호출하지만, CapNet에서는 사용자가 Node를 지정하지 않고 능력과 실행 조건만 요청한다. Core가 신뢰·자격·가용성을 확인해 실행 자원을 선택하고, 결과뿐 아니라 실제 실행 경로(증적)를 남긴다. 1) 능력=계약 — 입출력 스키마·전처리·실행 조건. 6종 능력이 같은 사슬 통과(image.classify, text.classify, image.embed, text.embed, timeseries.forecast, table.extract). 2) DB가 라우팅 강제 — privacy_rank 등 복합 FK. team 작업→public Node 배정 불가. 3) 살아 있는 Node만 — liveness·부하 기반 배정. 4) 실행 증적 — node-agent·가중치 해시·시각을 DB에 기록·조회. 5) 입력 통제 — Core 중개·해시·MIME 검증. 자유 업로드 경로 없음. 6) 위반 6종 DB 거절 — 자격 없는 배정, 도메인·등급 위반, lease 중 강등, READY 중 가중치 교체, PASSED 무효화 등. 7) 품질 프로파일(선택) — image.classify@1 scratch 실측 acc=0.8500 · macro-F1=0.8344 PASSED. 상수·난수·스키마 위반 Agent는 전부 FAILED.

기대효과 및 활용분야	【기대효과 및 활용분야】 조직 내부의 GPU PC·서버를 Node로 연결하면, 사용자는 기기 주소나 모델을 지정하지 않고 필요한 AI 능력(Capability)만 요청해 작업을 수행할 수 있다. 향후에는 유휴 PC·GPU를 풀로 넓히되, Core가 신뢰 도메인·연산 등급·Agent 자격·Node 상태를 확인해 자원을 고르는 방식을 유지한다. 실행 Node·Agent·가중치 등 경로가 증적으로 남으므로, 데이터 외부 반출이 어려운 기관·기업의 내부 AI 실행 환경과 감사·관리에 활용할 수 있다.
기타	【프로젝트의 혁신성 및 차별성】 기존 AI 서비스는 일반적으로 애플리케이션이 특정 AI 실행 서버와 API를 직접 연결하고, 실행 조건과 자원 선택에 대한 판단도 애플리케이션 로직에 의존한다. 이러한 구조에서는 실행 조건이 변경되거나 새로운 정책이 추가될 경우 애플리케이션의 수정과 배포가 필요하며, 모든 호출 경로에서 동일한 규칙이 적용되는지 별도로 확인해야 한다. CapNet은 AI 실행 자원의 선택과 실행 조건을 별도의 실행계층으로 분리하고, 라우팅에 필요한 불변식을 애플리케이션 로직만으로 판단하지 않도록 PostgreSQL의 제약조건과 데이터 모델로 강제했다. MVP에서는 이러한 설계가 실제로 동작하는지를 검증하기 위해 위반 사례 14종을 실측하고, 그중 6종을 재현 스크립트로 고정하여 데이터베이스가 실제로 잘못된 배정을 거부하는 것을 확인했다. 또한 사용자가 실행 기기에 직접 접속하지 않고 Core를 통해 작업을 전달하며, 입력 데이터 역시 Core가 통제된 경로로 전달하도록 구성했다. 【한계점 및 향후 발전 로드맵】 기기가 데이터를 남기지 않는다는 보장은 하지 않는다(TEE 없이 원리적으로 불가). 보장하는 것은 승인하지 않은 신뢰 도메인으로 라우팅되지 않는다는 것과 증적이 남는다는 것이다. 현재는 공개 실행 네트워크가 아니라 조직 내부 신뢰 플릿 검증 단계다. 골든셋 정적·공개 한계와 A/B 등가 한계는 문서화했다. 다음: 홀드아웃·회전 프로브·조직 단위 플릿. 【소감 및 후기】 이번 프로젝트를 진행하면서 AI 실행 자원의 문제는 단순히 더 많은 GPU를 확보하는 것이 아니라, 어떤 작업을 어떤 자원에서 실행할 수 있는지를 안전하게 결정하고 그 과정을 관리하는 것이라는 점을 확인했다. 처음에는 유휴 GPU와 PC를 연결해 활용하는 것에 초점을 두었지만, 구현 과정에서 자원 선택의 기준, Agent의 자격, 신뢰 경계, 실행 상태, 증적까지 함께 관리해야 실제 실행 계층으로 활용할 수 있다는 것을 배웠다.

붙임1

SBOM(소프트웨어 자재명세서)

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	psycopg	3.2.9	LGPL-3.0	https://github.com/psycopg/psycopg	PostgreSQL 드라이버
2	psycopg-pool	3.3.1	LGPL-3.0	https://github.com/psycopg/psycopg	커넥션 풀
3	FastAPI	0.116.1	MIT	https://github.com/tiangolo/fastapi	Core·Node HTTP API
4	uvicorn	0.35.0	BSD-3-Clause	https://github.com/encode/uvicorn	ASGI 서버
5	pydantic-settings	2.10.1	MIT	https://github.com/pydantic/pydantic-settings	환경변수 설정
6	safetensors	0.8.0	Apache-2.0	https://github.com/huggingface/safetensors	가중치 로드(pickle 거부)
7	numpy	2.4.6	BSD-3-Clause	https://github.com/numpy/numpy	텐서·배열 백엔드
8	Pillow	12.3.0	HPND-derived	https://github.com/python-pillow/Pillow	골든셋 JPEG 로드
9	torch	2.13.0+cpu	BSD-3-Clause	https://github.com/pytorch/pytorch	scratch 학습·추론
10	torchvision	0.28.0+cpu	BSD-3-Clause	https://github.com/pytorch/vision	32×32 변환

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 □ 표시)

- 유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동한 경우)
 □ 유형 2: 외부 모델 파인튜닝 (기존 공개 모델을 가져와 준비한 데이터셋으로 추가 미세조정한 경우)
 □ 유형 3: 자체 개발 모델 (기반 모델 없이 참가팀이 처음부터 가중치를 직접 전체 학습시킨 경우)
 판정: 기반 모델 없이 EuroSAT RGB만으로 가중치를 처음부터 학습 → 유형 3. 외부 상용 AI API 의존 없음(로컬 torch). 코딩 보조는 아래 4번 항목에 기재.

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음 (기반 모델 미사용. TinyEuroSAT을 EuroSAT RGB로 scratch 학습)	기반 모델 라이선스	해당 없음
--------------	---	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	EuroSAT RGB 배포판 · Zenodo record 7711810 · DOI 10.5281/zenodo.7711810 · 라이선스 MIT · EuroSAT_RGB.zip 27,000장(10클래스) · 원본 64×64 JPEG. 원본 zip은 저장소 미동봉(scripts/download_eurosat + archive_sha256 핀).
데이터 정제/ 가공 방법 요약	개인정보 없음(원격탐사 타일). 클래스 균등 스트라이드로 데모 골든 N=40 추출. 게이트·제품 계약은 32×32 resize. 학습은 전체 27,000장 · seed 20260806 · Agent A 80 epoch · Agent B 40 epoch(CPU, scratch, pretrained=false).
새로 생성된 가중치 공개 저장소 URL	https://github.com/gncorpseo-commits/capnet/blob/main/apps/node/weights/eurosat_scratch.safetensors https://github.com/gncorpseo-commits/capnet/blob/main/apps/node/weights/eurosat_scratch_b.safetensors 직접 다운로드(승인 없음): https://raw.githubusercontent.com/gncorpseo-commits/capnet/main/apps/node/weights/eurosat_scratch.safetensors https://raw.githubusercontent.com/gncorpseo-commits/capnet/main/apps/node/weights/eurosat_scratch_b.safetensors

가중치 파일 정보 및 배포방식	eurosat_scratch.safetensors / 전체 가중치 배포 (TinyEuroSAT CNN, 94,538 파라미터) / 370KB · sha256 74ca92224ff93f6cfab56265466d2c8ed11e0add4581c45a98169e92fb797b43 eurosat_scratch_b.safetensors / 전체 가중치 배포 (TinyEuroSATB CNN, 24,685 파라미터) / 98KB · sha256 3fbdde549459ba1e895ab8221ba1455b7840e1641b36b202c96420d410343b24 LoRA가 아닌 전체 가중치(유형 3). 메타는 각 *.meta.json.		
4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)			
직접 작성한 코드의 오픈소스 라이선스	Apache License 2.0	학습/추론 소스코드 공개 저장소 URL	https://github.com/gncorpseo-commits/capnet (apps/train/train_scratch.py, apps/node/app/infer.py, apps/node/app/score_gate.py)
상용 AI 보조도구 활용 여부 및 범위	코드 작성·디버깅·문서 정리 보조로 Cursor 및 Claude Code를 활용함. 전체 코드(약 16,903줄, py·sh·ps1·sql·yaml 기준) 중 AI 보조 작성분은 약 98%로 추정함. 스키마 제약·게이트 사슬·채점 규칙·판정 기준은 팀이 설계했고, AI 생성 코드는 팀이 동작 원리를 확인하고 실행 검증한 뒤 반영함.		